

S1 Text: Detailed methods and implementation for GPU-accelerated TMRCA inference

1 S1: The Sequentially Markov Coalescent for pairwise TMRCA

The Sequentially Markov Coalescent (SMC) provides a tractable approximation to the full ancestral recombination graph (ARG) by modeling the genealogical history along a pair of haplotypes as a Markov chain indexed by genomic position (?). Under the SMC, the time to the most recent common ancestor (TMRCA) at each locus evolves according to a hidden Markov model (HMM) whose hidden states are coalescence times and whose emissions are the observed pattern of genetic variation. This framework was first exploited for demographic inference by ? in the pairwise sequentially Markov coalescent (PSMC), and has since been extended in numerous directions (???). In this section we provide a complete mathematical description of the discrete-time pairwise SMC HMM that forms the foundation of our GPU implementation.

1.1 Time discretization

The continuous coalescence time $T \in [0, \infty)$ is approximated by a discrete set of K time points $\{t_0, t_1, \dots, t_{K-1}\}$. We adopt a quadratic spacing scheme that concentrates resolution in the recent past, where the posterior is most informative, while still covering deep coalescence times relevant to between-population divergence. Specifically, the k -th time point is defined as

$$t_k = t_{\max} \left(\frac{k}{K} \right)^2, \quad k = 0, 1, \dots, K-1, \quad (1)$$

where $t_{\max}$ is the maximum coalescence time considered (in units of generations scaled by $2N_e$) and K is the number of discrete states. The default parameterization uses $K = 32$ states. Quadratic spacing is motivated by the exponential decay of the coalescent prior: under a constant-size population, the density of coalescence times is $f(t) = \frac{1}{2N_e} \exp(-t/(2N_e))$, which places the bulk of its mass in the recent past. By spacing time points quadratically, we allocate more states to the region where the prior density is highest, thereby improving the resolution of the posterior precisely where it matters most. The interval associated with state k spans $[t_k, t_{k+1})$ for $k < K-1$, and $[t_{K-1}, \infty)$ for the last state.

1.2 Coalescent prior

The prior probability of coalescence in the k -th time interval, under a constant effective population size N_e , is obtained by integrating the exponential coalescent density over the interval $[t_k, t_{k+1})$:

$$q_k = \exp\left(-\frac{t_k}{2N_e}\right) - \exp\left(-\frac{t_{k+1}}{2N_e}\right). \quad (2)$$

For the final interval $k = K-1$, which extends to infinity, the prior simplifies to $q_{K-1} = \exp(-t_{K-1}/(2N_e))$. The vector $\mathbf{q} = (q_0, q_1, \dots, q_{K-1})$ sums to unity (up to the truncation at $t_{\max}$) and serves as both the initial state distribution $\pi_k = q_k$ and the target distribution for the recombination component of the transition matrix.

In the context of varying population size, one can replace $2N_e$ with a piecewise-constant or piecewise-linear function $\eta(t)$, leading to a more complex integral that depends on the cumulative coalescent rate $\Lambda(t) = \int_0^t \frac{1}{2N_e(s)} ds$. However, for the purpose of TMRCA estimation with a fixed demographic model, we pre-compute the prior vector $\mathbf{q}$ once and reuse it throughout the forward-backward algorithm.

1.3 Emission model

At each genomic site s , the observed data d_s is binary: $d_s = 1$ if the two haplotypes differ (heterozygous site), and $d_s = 0$ if they are identical (homozygous site). Under the infinite-sites mutation model, the probability of observing at least one mutation on the genealogical branch of total length $2t_k$ (since there are two lineages) follows a Poisson process with rate $2\mu t_k$, where μ is the per-site per-generation mutation rate. The emission probability for a heterozygous site is therefore

$$P(d_s = 1 \mid T = t_k) = 1 - \exp(-2\mu t_k). \quad (3)$$

Conversely, the probability of observing a homozygous site is $P(d_s = 0 \mid T = t_k) = \exp(-2\mu t_k)$.

In practice, we do not observe every base pair individually. Instead, we observe a sequence of variant positions separated by gaps of varying length. For a stretch of g consecutive homozygous sites (i.e., the inter-SNP gap between two consecutive heterozygous sites), the emission probability under state k is

$$P(\text{gap of length } g \mid T = t_k) = \exp(-2\mu t_k \cdot g), \quad (4)$$

which follows from the independence of mutations at different sites under the infinite-sites model. This gap emission is critical for computational efficiency: rather than iterating over every base pair, we can skip directly from one variant to the next, incorporating the gap emission as a multiplicative update to the forward variable. The gap emission effectively increases β in the Gamma parameterization (see Section S2), providing information that the TMRCA is not too small—otherwise more mutations would have been observed in the gap.

1.4 Transition model

The transition matrix $\mathbf{A}(r)$ governs the change in coalescence time between adjacent sites separated by a recombination probability r . Under the SMC approximation, a recombination event between two sites causes the genealogy at the downstream site to be redrawn from the coalescent prior, independently of the upstream genealogy (this is the key Markov approximation of the SMC). The transition probability from state k to state l is

$$A_{kl}(r) = \exp(-r t_k) \delta_{kl} + (1 - \exp(-r t_k)) q_l, \quad (5)$$

where δ_{kl} is the Kronecker delta. The first term represents the “stay” component: with probability $\exp(-r t_k)$, no recombination occurs on the branch of length t_k , and the coalescence time remains unchanged. The second term represents the “recombine” component: with probability $1 - \exp(-r t_k)$, a recombination event occurs, and the new coalescence time is drawn from the prior q_l . The dependence of the recombination probability on the current state t_k reflects the fact that longer branches (more ancient coalescence times) have more opportunity for recombination to occur.

1.5 Factorization for efficient GPU computation

The naive matrix-vector product for the forward algorithm requires $O(K^2)$ operations per site, since the transition matrix $\mathbf{A}(r)$ is a full $K \times K$ matrix. However, the rank-one-plus-diagonal structure of Eq. (5) admits a factorization that reduces this to $O(K)$ per site.

Consider the forward update $\alpha_l^{(s)} = \sum_{k=0}^{K-1} A_{kl}(r_s) \alpha_k^{(s-1)} e_k^{(s-1)}$, where $e_k^{(s-1)}$ is the emission probability at site $s-1$ in state k . Substituting Eq. (5):

$$\begin{aligned} \alpha_l^{(s)} &= \sum_{k=0}^{K-1} [\exp(-r_s t_k) \delta_{kl} + (1 - \exp(-r_s t_k)) q_l] \alpha_k^{(s-1)} e_k^{(s-1)} \\ &= \exp(-r_s t_l) \alpha_l^{(s-1)} e_l^{(s-1)} + q_l \sum_{k=0}^{K-1} (1 - \exp(-r_s t_k)) \alpha_k^{(s-1)} e_k^{(s-1)}. \end{aligned} \quad (6)$$

The sum $C_s = \sum_{k=0}^{K-1} (1 - \exp(-r_s t_k)) \alpha_k^{(s-1)} e_k^{(s-1)}$ is a scalar that can be computed in $O(K)$ time and shared across all target states l . Thus, the full forward update becomes $\alpha_l^{(s)} = \exp(-r_s t_l) \alpha_l^{(s-1)} e_l^{(s-1)} + q_l C_s$,

requiring only $O(K)$ operations per site. This factorization is essential for GPU efficiency, as it eliminates the need for shared-memory reduction across states within a single thread and allows each thread to process an entire pair of haplotypes independently through the forward and backward passes.

2 S2: The Gamma-SMC approximation

While the discrete HMM described in Section S1 provides an exact (up to discretization) posterior over coalescence times, its state space of $K = 32$ discrete bins imposes a non-trivial memory and computational burden when processing millions of sites across thousands of pairs. The Gamma-SMC approximation, introduced by ?, replaces the discrete state distribution with a continuous Gamma distribution, dramatically reducing the per-site state from K floating-point values to just two parameters (α, β) .

2.1 The Gamma posterior parameterization

At any genomic position s , the posterior distribution over the coalescence time T is approximated by a Gamma distribution:

$$T \mid \text{data up to site } s \sim \text{Gamma}(\alpha_s, \beta_s), \quad (7)$$

with density $f(t; \alpha, \beta) = \frac{\beta^\alpha}{\Gamma(\alpha)} t^{\alpha-1} e^{-\beta t}$ for $t > 0$. The posterior mean is $\mathbb{E}[T] = \alpha/\beta$ and the coefficient of variation is $\text{CV}[T] = 1/\sqrt{\alpha}$. The two-parameter representation captures both the location and the uncertainty of the coalescence time estimate.

The key insight enabling the Gamma-SMC is that the Gamma distribution is conjugate to the Poisson likelihood that arises from the infinite-sites mutation model. Under this model, the number of mutations on a branch of length $2t$ follows $\text{Poisson}(2\mu t)$, and the Gamma prior on t leads to a Gamma posterior after observing mutations.

2.2 Conjugate updates for mutations

When a heterozygous site is encountered at position s (i.e., a mutation is observed), the Bayesian update with a Poisson likelihood yields:

$$\alpha_s = \alpha_{s-1} + 1, \quad \beta_s = \beta_{s-1}. \quad (8)$$

This elegant update follows from the conjugacy of the Gamma prior with the Poisson likelihood. Observing one mutation increments the shape parameter by exactly one, while leaving the rate parameter unchanged. The posterior mean shifts from α_{s-1}/β_{s-1} to $(\alpha_{s-1} + 1)/\beta_{s-1}$, reflecting the increased estimate of TMRCA (more mutations imply a longer branch), while the coefficient of variation decreases from $1/\sqrt{\alpha_{s-1}}$ to $1/\sqrt{\alpha_{s-1} + 1}$, reflecting the reduced uncertainty (the observation provides additional information about the true coalescence time).

2.3 Gap emission as conjugate update

For a gap of g base pairs with no observed mutations, the likelihood contribution is $P(\text{no mutations in gap} \mid T = t) = \exp(-2\mu g t)$, which is an exponential function of t . Multiplying the Gamma density by this likelihood:

$$\begin{aligned} f(t; \alpha, \beta) \cdot \exp(-2\mu g t) &\propto t^{\alpha-1} \exp(-\beta t) \cdot \exp(-2\mu g t) \\ &= t^{\alpha-1} \exp(-(\beta + 2\mu g) t), \end{aligned} \quad (9)$$

which is again a Gamma density with updated parameters:

$$\alpha_s = \alpha_{s-1}, \quad \beta_s = \beta_{s-1} + 2\mu g. \quad (10)$$

The shape parameter is unchanged (no new mutation events), but the rate parameter increases, reflecting the information that the absence of mutations over a long stretch of sequence constrains the TMRCA to be small. The posterior mean decreases from α/β to $\alpha/(\beta + 2\mu g)$, pulling the estimate toward zero. Gaps thus serve as a regularizer, preventing the TMRCA estimate from drifting to unreasonably large values in mutation-sparse regions.

2.4 Recombination transition via moment matching

The recombination transition is the most complex step, as the mixture of the current posterior with the coalescent prior does not yield a Gamma distribution in closed form. Let $p_{\text{rec}} = 1 - \exp(-r \cdot g \cdot \mathbb{E}[T])$ be the probability that a recombination event occurs in a gap of g base pairs, where r is the per-base-pair recombination rate and $\mathbb{E}[T] = \alpha/\beta$ is the current posterior mean of the coalescence time. After a potential recombination, the new coalescence time is drawn from the mixture:

$$T' \sim (1 - p_{\text{rec}}) \cdot \text{Gamma}(\alpha, \beta) + p_{\text{rec}} \cdot \text{Coalescent prior}, \quad (11)$$

where the coalescent prior is $f_{\text{prior}}(t) = \frac{1}{2N_e} \exp(-t/(2N_e))$, i.e., an Exponential distribution with rate $\lambda = 1/(2N_e)$, which is itself a $\text{Gamma}(1, \lambda)$.

Since this mixture is not Gamma-distributed, we approximate it by a Gamma distribution whose first two moments match those of the mixture. The mean of the mixture is

$$\mu' = (1 - p_{\text{rec}}) \cdot \frac{\alpha}{\beta} + p_{\text{rec}} \cdot 2N_e, \quad (12)$$

and the second moment is

$$\mathbb{E}[(T')^2] = (1 - p_{\text{rec}}) \cdot \frac{\alpha(\alpha + 1)}{\beta^2} + p_{\text{rec}} \cdot (2N_e)^2 \cdot 2. \quad (13)$$

The variance is $\sigma'^2 = \mathbb{E}[(T')^2] - (\mu')^2$, and the moment-matched Gamma parameters are

$$\alpha' = \frac{(\mu')^2}{\sigma'^2}, \quad \beta' = \frac{\mu'}{\sigma'^2}. \quad (14)$$

This moment-matching step introduces an approximation error that accumulates over the sequence. However, the error is small when the recombination probability per inter-SNP gap is low (as is typical in human genetics), because the mixture is then dominated by the current posterior component and is nearly Gamma-distributed.

2.5 Linearization of the recombination probability

A key computational simplification arises from the observation that, in human genetics, the product $\rho \cdot g$ (recombination rate times gap length in base pairs) is typically very small. The per-base-pair recombination rate in humans is approximately $\rho \approx 10^{-8}$, and the average spacing between SNPs in a typical dataset is on the order of $g \approx 300$ bp (for common variants). Thus, $x = \rho \cdot g \cdot \mathbb{E}[T]$ is on the order of 10^{-6} to 10^{-4} when $\mathbb{E}[T]$ is in the range of 10^2 to 10^4 generations.

For such small x , the exponential function can be linearized:

$$1 - \exp(-x) \approx x, \quad \text{for } x \ll 1. \quad (15)$$

The relative error of this approximation is bounded by

$$\frac{|x - (1 - e^{-x})|}{1 - e^{-x}} = \frac{x - 1 + e^{-x}}{1 - e^{-x}}. \quad (16)$$

By Taylor expansion, $1 - e^{-x} = x - x^2/2 + x^3/6 - \dots$ and $x - (1 - e^{-x}) = x^2/2 - x^3/6 + \dots$, so the relative error is approximately $x/2$ for small x . For $x < 0.01$, the relative error is bounded by 0.005, i.e., less than 0.5%. For typical human genetic parameters where $\rho \cdot g \cdot \mathbb{E}[T] < 0.01$, the relative error is less than 5×10^{-5} , making the linearization essentially exact for practical purposes. More precisely, setting $x = \rho \cdot g$ where $\rho = 10^{-8}$ and $g = 300$ gives $x = 3 \times 10^{-6}$, and even multiplied by a generous $\mathbb{E}[T] = 10^4$, we get $x' = 0.03$, with relative error ≈ 0.015 .

This linearization avoids the evaluation of the exponential function—a relatively expensive operation on GPU hardware—and replaces it with a single multiply. In our implementation, we use the function `fast_one_minus_exp_neg(x)` which returns x directly when $x < 0.01$ and falls back to $1 - \text{__expf}(-x)$ (the CUDA fast-math single-precision exponential) otherwise.

3 S3: Flow field transition model

The moment-matching procedure described in Section S2 requires computing the first two moments of a mixture distribution at every inter-SNP gap. While this is tractable, it involves multiple multiplications and divisions that become a bottleneck when processing millions of sites. ? introduced an elegant alternative: precomputing the recombination transition as a two-dimensional flow field over the space of Gamma distribution parameters, enabling the transition to be applied via simple table lookup and interpolation.

3.1 State space representation

The state of the Gamma posterior at any genomic position is fully characterized by the two parameters (α, β) , or equivalently by the posterior mean $m = \alpha/\beta$ and coefficient of variation $CV = 1/\sqrt{\alpha}$. For numerical stability and uniform grid resolution, the flow field is defined over the log-transformed coordinates:

$$u = \log_{10}(m), \quad v = \log_{10}(CV). \quad (17)$$

The mean m represents the estimated TMRCA (in coalescent units), while the CV captures the posterior uncertainty: $CV = 1$ corresponds to an exponential distribution (maximum uncertainty), and $CV \rightarrow 0$ corresponds to a point mass (complete certainty).

3.2 Grid specification

The flow field is discretized on a regular grid in (u, v) space:

- **Mean axis:** $u = \log_{10}(m)$ ranges from -5 to 2 in 51 evenly spaced points, corresponding to TMRCA values from 10^{-5} to 10^2 in coalescent units ($2N_e$ generations). This range covers coalescence times from essentially zero to 200,000 generations (for $N_e = 10^4$), spanning the timescale relevant to both within-population and between-species divergence.
- **CV axis:** $v = \log_{10}(CV)$ ranges from -2 to 0 in 50 evenly spaced points, corresponding to CV values from 0.01 (highly concentrated posterior, $\alpha = 10^4$) to 1.0 (exponential distribution, $\alpha = 1$). Values of CV above 1 would correspond to $\alpha < 1$, which implies a mode at zero and is generally not encountered in practice after observing data.

The total grid size is $51 \times 50 = 2550$ points, a remarkably compact representation of the entire transition dynamics.

3.3 Flow field construction

At each grid point (u_i, v_j) , the flow field stores a displacement vector $(\Delta u, \Delta v)$ that describes how the Gamma parameters change due to one unit of recombination (specifically, a recombination event with $\rho \cdot \Delta x = 1$, where Δx is measured in genetic map distance). The displacement is computed by:

1. Converting (u_i, v_j) back to Gamma parameters: $m = 10^{u_i}$, $CV = 10^{v_j}$, $\alpha = 1/CV^2$, $\beta = \alpha/m$.
2. Computing the mixture moments after a unit recombination step using the exact moment-matching formulas from Section S2.
3. Converting the updated parameters (m', CV') back to log space: $u' = \log_{10}(m')$, $v' = \log_{10}(CV')$.
4. Storing the displacement: $\Delta u = u' - u_i$, $\Delta v = v' - v_j$.

The flow field thus encodes the infinitesimal generator of the recombination dynamics in the log-parameter space. Because the coalescent prior acts as an attractor (all states eventually converge to the prior under repeated recombination), the flow field has a single stable fixed point at $(u^*, v^*) = (\log_{10}(2N_e), 0)$, corresponding to the exponential coalescent prior.

3.4 Bilinear interpolation

For a Gamma state (u, v) that falls between grid points, the flow displacement is computed via standard bilinear interpolation. Let (i, j) be the grid cell containing (u, v) , and let $s = (u - u_i)/(u_{i+1} - u_i)$ and $t = (v - v_j)/(v_{j+1} - v_j)$ be the fractional coordinates within the cell. The interpolated displacement is:

$$\Delta u(u, v) = (1 - s)(1 - t) \Delta u_{ij} + s(1 - t) \Delta u_{i+1,j} + (1 - s)t \Delta u_{i,j+1} + st \Delta u_{i+1,j+1}, \quad (18)$$

$$\Delta v(u, v) = (1 - s)(1 - t) \Delta v_{ij} + s(1 - t) \Delta v_{i+1,j} + (1 - s)t \Delta v_{i,j+1} + st \Delta v_{i+1,j+1}. \quad (19)$$

Bilinear interpolation is exact for affine flow fields and introduces only second-order errors for smooth non-linear fields, which is sufficient for the slowly varying recombination dynamics.

3.5 Multi-step cache and adaptive sub-stepping

To further accelerate the transition computation, the implementation precomputes a multi-step cache that stores the cumulative flow result for n consecutive base pairs of recombination, for $n = 1, 2, \dots, n_{\max}$. This allows inter-SNP gaps of up to $n_{\max}$ base pairs to be traversed with a single cache lookup rather than n sequential flow steps.

However, for large gaps, the accumulated recombination distance $\rho \cdot g$ may be large enough that a single flow step is inaccurate (the displacement vector is only valid for infinitesimal steps). To maintain accuracy, the implementation enforces a maximum step size of `MAX_STEP_RHO` = 0.1 in genetic map distance. When the total recombination distance for a gap exceeds this threshold, the gap is broken into multiple sub-steps, each of size at most `MAX_STEP_RHO`, and the flow is integrated sequentially across these sub-steps. The adaptive sub-stepping ensures that the linearization error (Section S2.4) remains negligible even across recombination hotspots or exceptionally long gaps between variants.

In practice, for a uniform recombination rate of $\rho = 10^{-8}/\text{bp}$, the threshold corresponds to a gap of 10^7 bp (10 Mb), which is far larger than any typical inter-SNP distance in whole-genome sequencing data. Thus, sub-stepping is rarely triggered, and the multi-step cache provides the dominant acceleration.

4 S4: GPU kernel architecture

The core computational kernel of `tmrca.cu` processes all haplotype pairs in parallel on the GPU, with one CUDA thread assigned to each pair. Each thread independently executes the forward and backward passes of the Gamma-SMC algorithm, storing intermediate results in global memory for the subsequent smoothing step. This section describes the kernel architecture, memory layout, and data encoding in detail.

4.1 Thread assignment and parallelism model

For n haploid samples, there are $\binom{n}{2}$ unique pairs. Each pair is assigned to a single CUDA thread, identified by a global thread index `pid` $\in \{0, 1, \dots, n_{\text{pairs}} - 1\}$. The pair index is mapped to haplotype indices (i, j) via a standard combinatorial unranking formula. All threads execute the same code path (no warp divergence beyond the per-site heterozygosity check), iterating over all S variant sites sequentially. The parallelism is thus over pairs, not over sites, which is the natural decomposition for this problem since the forward-backward algorithm requires sequential processing along the genome but is independent across pairs.

This pair-parallel architecture maps efficiently to GPU hardware. For $n = 200$ haplotypes (100 diploid individuals), $n_{\text{pairs}} = 19,900$, which provides ample parallelism to saturate even large GPUs. Each thread has a modest register footprint (the Gamma state requires only two floats plus a few temporaries), and the primary memory access pattern is a sequential scan through the genotype and position arrays, enabling effective use of the L1/L2 cache hierarchy.

4.2 Bitpacked genotype representation

Genotype data is stored in a bitpacked format where each `uint64` word encodes 64 consecutive variant sites for a single haplotype, with the least significant bit (LSB) corresponding to the lowest-indexed site in the word. For haplotype i and word w (covering sites $64w$ through $64w + 63$), the genotype is stored

in `geno_buf[i * n_words + w]`, where $n_{\text{words}} = \lceil S/64 \rceil$. A set bit indicates the derived allele (mutation present), and a cleared bit indicates the ancestral allele.

Bitpacking provides two critical advantages. First, it reduces memory consumption and bandwidth by a factor of 64 compared to byte-per-site representations, which is essential when the genotype matrix must reside in GPU global memory. For $n = 200$ haplotypes and $S = 10^6$ sites, the bitpacked representation requires $200 \times 15,625 \times 8 = 25$ MB, compared to 200 MB for a byte representation. Second, it enables the use of hardware-accelerated bitwise operations (`XOR`, `__popc11`) to rapidly identify heterozygous sites, as described below.

4.3 Pre-XOR kernel

Before the forward-backward passes, a pre-processing kernel computes the XOR of each pair's genotype arrays and stores the result in a dedicated buffer:

$$\text{xor_buf}[\text{pid} * n_{\text{words}} + w] = \text{geno_buf}[i * n_{\text{words}} + w] \oplus \text{geno_buf}[j * n_{\text{words}} + w], \quad (20)$$

where (i, j) are the haplotype indices for pair `pid` and $\oplus$ denotes the bitwise XOR operation. The `precompute_xor_kernel` is launched with $n_{\text{pairs}} \times n_{\text{words}}$ threads, with each thread computing a single XOR word.

The pre-XOR buffer transforms two scattered global memory reads (for haplotypes i and j , which may be widely separated in memory) into a single sequential read during the forward pass. Since the forward pass iterates over all sites in order, the sequential access pattern allows the GPU memory controller to coalesce reads across threads (all threads access the same word index w at the same time, with different pair indices `pid`), achieving near-peak memory bandwidth. The buffer size is $n_{\text{pairs}} \times n_{\text{words}} \times 8$ bytes; for $n_{\text{pairs}} = 19,900$ and $n_{\text{words}} = 15,625$ (1M sites), this is approximately 2.4 GB.

4.4 Forward pass

The forward pass iterates over sites $s = 0, 1, \dots, S - 1$, maintaining the Gamma posterior (α_s, β_s) for each pair. At each site, the following steps are performed:

1. **Cache advance (flow field lookup):** The recombination transition is applied for the gap between the previous site and the current site. The gap length $g = \text{pos}[s] - \text{pos}[s - 1]$ (in base pairs) determines the recombination distance. The flow field lookup, as described in Section S3, updates the Gamma parameters to account for both the recombination transition and the gap emission (no mutations observed in the gap). In the log-parameter representation, this amounts to:

$$u_s = u_{s-1} + \Delta u(u_{s-1}, v_{s-1}; g), \quad (21)$$

$$v_s = v_{s-1} + \Delta v(u_{s-1}, v_{s-1}; g), \quad (22)$$

where the displacements are obtained from the multi-step cache or adaptive sub-stepping.

2. **Heterozygous emission:** If site s is heterozygous for this pair (determined by checking the appropriate bit in the XOR buffer), the shape parameter is incremented: $\alpha_s += 1$. In the log-parameter representation, this corresponds to an update of both u and v :

$$\alpha = 1/(10^{2v})^2 = 10^{-2v}, \quad (23)$$

$$\alpha' = \alpha + 1, \quad (24)$$

$$u' = u + \log_{10}(\alpha'/\alpha) = u + \log_{10}(1 + 10^{2v}), \quad (25)$$

$$v' = -\frac{1}{2} \log_{10}(\alpha') = v - \frac{1}{2} \log_{10}(1 + 10^{2v}). \quad (26)$$

3. **Store forward state:** The current (u_s, v_s) values are written to the forward buffer for later use in the backward-smoothing pass:

$$\text{fwd_mean}[s \times n_{\text{pairs}} + \text{pid}] = u_s, \quad (27)$$

$$\text{fwd_cv}[s \times n_{\text{pairs}} + \text{pid}] = v_s. \quad (28)$$

The site-major layout (site index varies fastest in the outer dimension) ensures that when all threads write their results for site s simultaneously, the writes are coalesced: thread 0 writes to address $s \times n_{\text{pairs}} + 0$, thread 1 writes to $s \times n_{\text{pairs}} + 1$, etc., forming a contiguous block.

4.5 Backward pass and smoothing

The backward pass iterates in reverse, from site $s = S - 1$ down to $s = 0$, maintaining a backward Gamma state $(\alpha_s^{\text{bwd}}, \beta_s^{\text{bwd}})$. At each site, the backward state is combined with the stored forward state to produce the smoothed (posterior) estimate. The combination of forward and backward Gamma states is performed via a product-of-experts approximation: since both the forward and backward messages are Gamma-distributed, their product is also Gamma-distributed (up to normalization), with parameters:

$$\alpha_s^{\text{smooth}} = \alpha_s^{\text{fwd}} + \alpha_s^{\text{bwd}} - 1, \quad (29)$$

$$\beta_s^{\text{smooth}} = \beta_s^{\text{fwd}} + \beta_s^{\text{bwd}} - \lambda, \quad (30)$$

where $\lambda = 1/(2N_e)$ is the rate of the coalescent prior. The subtraction of 1 from α and λ from β corrects for the double-counting of the prior, which is implicitly included in both the forward and backward initializations. The smoothed posterior mean at site s for pair `pid` is then:

$$m_s^{\text{smooth}} = \frac{\alpha_s^{\text{smooth}}}{\beta_s^{\text{smooth}}}. \quad (31)$$

Rather than storing the full smoothed posterior for all sites and pairs (which would require $2 \times S \times n_{\text{pairs}}$ floats), the backward pass fuses the smoothing computation with a reduction over pairs. At each site s , each thread computes m_s^{smooth} and adds it to a shared accumulator:

$$\text{atomicAdd}(\&\text{site_sum}[s], m_s^{\text{smooth}}). \quad (32)$$

After all threads complete, `site_sum[s]` contains the sum of smoothed posterior means across all pairs, which can be divided by n_{pairs} to obtain the average TMRCA estimate at site s .

4.6 Memory layout of the forward buffer

The forward buffer `fwd_buf` is a contiguous allocation of $2 \times S \times n_{\text{pairs}}$ single-precision floats, logically partitioned into two sub-arrays:

- **fwd_mean**: the first $S \times n_{\text{pairs}}$ floats, storing $u_s = \log_{10}(\text{mean})$ in site-major order.
- **fwd_cv**: the next $S \times n_{\text{pairs}}$ floats, storing $v_s = \log_{10}(\text{CV})$ in site-major order.

The site-major layout is chosen to maximize memory coalescing during both the forward pass (writes) and the backward pass (reads). During the forward pass, all threads write to consecutive addresses within the same site’s block. During the backward pass, all threads read from consecutive addresses as they process the same site in lockstep. This coalescing is critical for achieving high effective bandwidth on GPU architectures, where the memory controller services requests in 128-byte aligned segments.

For $n_{\text{pairs}} = 19,900$ and $S = 15,000$ sites, the forward buffer requires $2 \times 15,000 \times 19,900 \times 4 = 2.38$ GB, which fits comfortably in the 80 GB of the A100’s HBM2e memory.

5 S5: Optimization details

This section documents several implementation-level optimizations that, while individually modest, collectively contribute to the observed 200–12,000 $\times$ speedup over the reference Schweiger `gamma_smc` implementation.

5.1 Linearized recombination

As discussed in Section S2.4, the recombination probability $1 - \exp(-x)$ can be replaced by the linear approximation x when x is small. Our implementation uses the function:

```
__device__ float fast_one_minus_exp_neg(float x) {
    return (x < 0.01f) ? x : (1.0f - __expf(-x));
}
```

The threshold of 0.01 is chosen to ensure that the relative error is below 0.5% in all cases. In human genetics with a per-base-pair recombination rate of $\rho \approx 10^{-8}$ and average SNP spacing of ~ 300 bp, the argument $x = \rho \cdot g \approx 3 \times 10^{-6}$ is far below the threshold, so the linear branch is taken at virtually every site. This eliminates the cost of the `__expf` intrinsic (approximately 20 clock cycles on Ampere architecture) and replaces it with a single floating-point multiply (4 clock cycles), providing a $5\times$ speedup for this micro-operation. Since the recombination step is executed once per site per pair, and the forward-backward algorithm visits each site twice (forward and backward), the cumulative savings across the entire genome are substantial.

5.2 Pre-XOR buffer

The pre-XOR buffer, described in Section S4.3, eliminates redundant global memory accesses during the forward pass. Without the pre-XOR buffer, each thread would need to read two genotype words per 64 sites (one for each haplotype in the pair), compute the XOR, and then process the result. The two genotype reads are scattered in memory (haplotype i and haplotype j may be separated by $n_{\text{words}} \times (j - i)$ words), leading to poor cache utilization when n is large.

With the pre-XOR buffer, the XOR is computed once in a separate kernel (which achieves near-peak bandwidth due to its simple access pattern), and the forward pass reads only a single contiguous array. The buffer size is $n_{\text{pairs}} \times n_{\text{words}} \times 8$ bytes. For the benchmark configuration of $n = 200$ haplotypes ($n_{\text{pairs}} = 19,900$) and $S = 1,000,000$ sites ($n_{\text{words}} = 15,625$), the buffer requires $19,900 \times 15,625 \times 8 = 2.49$ GB. While this is a significant memory allocation, it is a worthwhile trade-off given the 80 GB available on the A100 and the substantial bandwidth savings during the compute-bound forward pass.

5.3 Fused backward pass and reduction

The backward pass and the pair-wise reduction are fused into a single kernel to avoid materializing the full smoothed posterior array. At each site s , each thread computes the smoothed mean m_s^{smooth} (Eqs. 29–30) and immediately performs an atomic addition to the per-site accumulator:

$$\text{atomicAdd}(\&\text{site_sum}[s], m_s^{\text{smooth}}). \quad (33)$$

For the benchmark configuration with $n_{\text{pairs}} = 19,900$ and $S = 15,000$ sites, this results in $19,900 \times 15,000 = 298.5 \times 10^6$ atomic additions over the course of the backward pass. The NVIDIA A100 achieves an atomic throughput of approximately 10^8 to 10^9 32-bit atomic operations per second (depending on contention), so the total time for atomic operations is on the order of 0.3–3 ms. This is negligible compared to the overall kernel execution time, which is dominated by the flow field lookups and memory accesses.

The fusion eliminates the need to allocate a $S \times n_{\text{pairs}}$ output array for smoothed means, saving $15,000 \times 19,900 \times 4 = 1.19$ GB of GPU memory and the associated memory bandwidth for writing and reading this array.

5.4 Persistent FlowContext

The `FlowContext` object encapsulates all GPU-resident data structures needed for the forward-backward computation: bitpacked genotypes, variant positions, the flow field lookup table, the multi-step cache, and the forward buffer. By keeping the `FlowContext` persistent across multiple calls (e.g., when processing different chromosomes or updating parameters in an optimization loop), we avoid the overhead of repeated GPU memory allocation and data transfer.

For the benchmark configuration, the total GPU memory managed by the `FlowContext` is approximately 2.4 GB (forward buffer) + 2.5 GB (XOR buffer) + 25 MB (genotypes) + 0.5 MB (positions) + 80 KB (flow field) $\approx$ 4.9 GB. Allocating this memory with `cudaMalloc` takes approximately 30 ms on the A100 (due to the overhead of the CUDA memory allocator for large allocations), and transferring the data from host to device adds another 10–20 ms at PCIe Gen4 bandwidth. By amortizing this cost across multiple invocations, the persistent context reduces the effective overhead to near zero.

5.5 Pair chunking for large sample sizes

When the number of pairs n_{pairs} is too large for the forward buffer to fit in GPU memory, the pairs are processed in chunks. For n haplotypes and chunk size C , the computation proceeds as follows:

1. Partition the n_{pairs} pairs into $\lceil n_{\text{pairs}}/C \rceil$ chunks of at most C pairs each.
2. For each chunk, allocate a forward buffer of size $2 \times S \times C$ floats, run the forward and backward passes, and accumulate the per-site sums into a host-side buffer.
3. After all chunks are processed, divide the accumulated sums by n_{pairs} to obtain the final per-site mean estimates.

The chunk size C is chosen to maximize GPU utilization while fitting within the available memory. For the A100 with 80 GB, and a forward buffer of $2 \times S \times C \times 4$ bytes, plus the XOR buffer of $C \times n_{\text{words}} \times 8$ bytes, the constraint is:

$$(8S + 8n_{\text{words}}) \times C \leq M_{\text{avail}}, \quad (34)$$

where M_{avail} is the available GPU memory after accounting for genotype data and other fixed allocations. For $S = 10^6$ and $n_{\text{words}} = 15,625$, the per-pair memory is $(8 \times 10^6 + 8 \times 15,625) \approx 8.1$ MB, allowing approximately $C = 9,000$ pairs per chunk with 73 GB available. For $n = 1,000$ haplotypes ($n_{\text{pairs}} = 499,500$), this requires approximately 56 chunks, each processed independently.

The overhead of chunking is minimal: the additional host-device memory transfers for the XOR buffer recomputation are overlapped with kernel execution using CUDA streams, and the reduction across chunks is a simple element-wise addition on the host.

6 S6: Schweiger gamma_smc output format

The reference implementation by ? outputs posterior estimates in a custom binary format compressed with the Zstandard (zstd) algorithm. Understanding this format is necessary for validating `tmrca.cu` against the reference and for downstream analysis tools that consume `gamma_smc` output. This section documents the format in detail.

6.1 Meta file

Each output consists of two files: a binary data file (with `.bin.zst` extension) and a JSON metadata file (with `.meta.json` extension). The metadata file contains:

- **num_pairs**: the total number of haplotype pairs in the output.
- **sequence_length**: the length of the genomic region analyzed (in base pairs).
- **chunk_size**: the number of pairs grouped into each output chunk, fixed at 8.
- **sample_names**: a list of diploid sample identifiers.
- **output_positions**: a list of genomic positions (0-indexed) at which posterior estimates are reported.

6.2 Binary data layout

The decompressed binary data is a four-dimensional array of single-precision (32-bit) floating-point values with shape:

$$[\text{n_chunks}, 2, \text{n_sites}, \text{chunk_size}], \quad (35)$$

where $\text{n_chunks} = \lceil \text{num_pairs} / \text{chunk_size} \rceil$ and $\text{n_sites} = |\text{output_positions}|$. The array is stored in row-major (C) order, so the `chunk_size` dimension varies fastest.

The second dimension of size 2 corresponds to two output channels:

- **Channel 0:** Posterior mean of the coalescence time, in scaled coalescent units. To convert to generations, multiply by $2N_e$.
- **Channel 1:** Posterior coefficient of variation (CV), providing a measure of uncertainty. A CV close to 0 indicates high confidence in the estimate, while a CV close to 1 indicates that the posterior is nearly as broad as the prior.

6.3 Pair ordering

The ordering of pairs in the output follows a specific convention tied to the diploid sample structure, which differs from a naive lexicographic ordering of haplotype indices. The convention is as follows:

Within-individual pairs are listed first. For each diploid individual A (with haplotypes $2A$ and $2A+1$), the within-individual pair $(2A, 2A+1)$ is included. These pairs appear in order of individual index.

Between-individual pairs follow, grouped by diploid pair. For diploid individuals $A < B$, the four haplotype pairs are ordered as:

1. $(2A, 2B)$: first haplotype of A with first haplotype of B ,
2. $(2A, 2B+1)$: first haplotype of A with second haplotype of B ,
3. $(2A+1, 2B)$: second haplotype of A with first haplotype of B ,
4. $(2A+1, 2B+1)$: second haplotype of A with second haplotype of B .

The diploid pairs are ordered lexicographically: $(0, 1), (0, 2), \dots, (0, n_{\text{diploid}} - 1), (1, 2), \dots$

This ordering is important for correctly matching `tmrca.cu` output to `gamma_smc` output when computing accuracy metrics. A mismatch in pair ordering would introduce spurious disagreement even if the underlying estimates are identical.

7 S7: Supplementary benchmark details

This section provides comprehensive details on the hardware, software, simulation parameters, and evaluation metrics used in the benchmark comparison between `tmrca.cu` and the reference `gamma_smc` implementation.

7.1 Hardware configuration

All benchmarks were executed on the `poppy.uoregon.edu` server, equipped with:

- **GPU:** NVIDIA A100 80GB PCIe (Ampere architecture, compute capability 8.0). The A100 features 6912 CUDA cores, 432 Tensor cores, 80 GB of HBM2e memory with 2.0 TB/s bandwidth, and a peak single-precision throughput of 19.5 TFLOPS.
- **CPU:** Sufficient for data preparation and `gamma_smc` execution (the reference implementation is CPU-only).
- **CUDA runtime:** Version 12.2, with driver version supporting all required features including cooperative groups and atomic operations on global memory.

7.2 Software versions and compilation

- **tmrca.cu**: Compiled with `nvcc` targeting `sm_80` (A100 native architecture), using `gcc 12.4` as the host compiler. The build system uses CMake (version 3.22+) with the Ninja generator for parallel compilation. Compiler flags include `-O3 -use_fast_math` for maximum performance, with `-lineinfo` for profiling support.
- **gamma_smc**: The binary release from the Schweiger et al. repository, compiled for `x86_64` Linux.
- **Flow field**: The file `default_flow_field.txt` from the Schweiger et al. repository, containing the precomputed 2D flow field described in Section S3.

7.3 Simulation parameters

Simulated datasets were generated using `msprime` version 1.2+ (?) with the following procedure:

1. **Ancestry simulation**: `msprime.sim_ancestry()` was called with a specified sample size, sequence length, recombination rate, and demographic model. The recombination rate was set to $\rho = 1.25 \times 10^{-8}$ per base pair per generation unless otherwise specified.
2. **Mutation simulation**: `msprime.sim_mutations()` was applied to the resulting tree sequence with a per-site per-generation mutation rate of $\mu = 1.25 \times 10^{-8}$ under the infinite-sites model (Jukes-Cantor).
3. **Demographic models**: We used both a constant-size population ($N_e = 10,000$) and the ? Out-of-Africa (OOA) three-population model as implemented in `stdpopsim` (?). The OOA model includes population size changes, bottlenecks, and migration, providing a realistic test case for the accuracy of TMRCA inference.

7.4 Timing methodology

Execution times were measured as the minimum of 5 independent repetitions, excluding an initial warmup call. The warmup call is necessary to account for one-time costs such as CUDA context initialization, JIT compilation of PTX to SASS, and initial memory allocation, which would inflate the timing of the first call. The minimum (rather than mean) is used to reduce the impact of system noise (e.g., OS scheduling, thermal throttling) and to measure the intrinsic performance of the algorithm.

For `tmrca.cu`, timing was measured using CUDA events (`cudaEventRecord` and `cudaEventElapsedTime`), which provide sub-millisecond precision and are synchronized with the GPU. The reported time includes only the GPU kernel execution and any necessary host-device memory transfers, excluding data loading from disk and output writing.

For `gamma_smc`, timing was measured using wall-clock time (`std::chrono::high_resolution_clock`), encompassing the full execution from input parsing to output writing. Since `gamma_smc` performs all computation on the CPU and includes I/O in its main loop, this provides a fair comparison of end-to-end performance.

7.5 Accuracy evaluation

The accuracy of TMRCA estimates was evaluated by comparing the inferred posterior mean at each site against the true TMRCA obtained from the `msprime` tree sequence. Since the true TMRCA and the estimated TMRCA both span several orders of magnitude, we evaluate accuracy in log space:

$$r = \text{Pearson}\left(\log\left(\max(\hat{T}_s, 10^{-10})\right), \log\left(\max(T_s^{\text{true}}, 10^{-10})\right)\right), \quad (36)$$

where $\hat{T}_s$ is the estimated TMRCA at site s and T_s^{true} is the true TMRCA. The $\max(\cdot, 10^{-10})$ floor prevents numerical issues with $\log(0)$ at sites where either the estimate or the truth is exactly zero (which can occur at the boundaries of the sequence or in regions of very recent coalescence).

The Pearson correlation in log space is a natural metric for this problem because it is invariant to multiplicative scaling (which would be absorbed by a global calibration factor) and emphasizes the ability to

distinguish between different orders of magnitude of coalescence time, which is the primary goal of TMRCA inference in population genetics applications.

We additionally report the root mean squared error (RMSE) in log space:

$$\text{RMSE}_{\log} = \sqrt{\frac{1}{S} \sum_{s=1}^S \left(\log(\hat{T}_s) - \log(T_s^{\text{true}}) \right)^2}, \quad (37)$$

and the fraction of sites where the estimate is within a factor of 2 of the truth: $|\log_2(\hat{T}_s/T_s^{\text{true}})| < 1$.

The speedup of tmrca.cu over gamma_smc is reported as the ratio of gamma_smc execution time to tmrca.cu execution time, ranging from approximately $200\times$ for small inputs (where GPU launch overhead is a significant fraction of total time) to approximately $12,000\times$ for large inputs (where the GPU's massive parallelism is fully exploited and the amortized overhead is negligible).